
CHAPTER 1

INTRODUCTION

1.1 About the Company

Inventeron Technologies and Business Solutions LLP made a humble beginning in 2013. Since inception, Inventeron has focused on serving people through quality-driven engineering, reliable delivery, and customer-centric execution. The organization has evolved into a trusted end-to-end engineering partner delivering software, hardware, embedded systems, IoT, electric mobility, Artificial Intelligence, Data Science, Machine Learning, and analytics solutions from concept and prototype to scalable production systems.

Inventeron Technologies and Business Solutions LLP operates across multiple domains including software engineering, hardware integration, embedded systems, Internet of Things (IoT), electric vehicle (EV) technologies, and Artificial Intelligence solutions. The company provides end-to-end engineering services, supporting projects from concept development and prototyping to full-scale production systems. By combining expertise in software, hardware, and advanced analytics, Inventeron delivers integrated solutions that reduce development time, improve efficiency, and ensure reliable system performance across industries.

Through its training and internship initiatives, Inventeron enables students and aspiring professionals to gain practical exposure to emerging technologies such as web development, cloud computing, data science, machine learning, IoT, and embedded systems. The organization focuses on bridging the gap between academic knowledge and industry requirements by offering hands-on learning experiences and real-world project involvement. With a strong emphasis on innovation, research and development, and industry collaboration, Inventeron equips learners with the technical skills and practical insights needed to succeed in today's rapidly evolving, technology-driven landscape.

1.1.1 Services Offered

Inventeron Technologies and Business Solutions LLP provides a comprehensive range of services across advanced engineering, software development, and emerging technology domains. The key service areas include:

1. Engineering & Technology Services

- Embedded systems development and hardware integration.
- Internet of Things (IoT) solutions and device connectivity.
- Electric Vehicle (EV) system integration, testing, and validation.
- End-to-end product development from concept to scalable production systems.

2. IT & Software Services

- Web and enterprise application development.
- Backend systems, APIs, and custom software platforms.
- Data engineering, analytics, and system integration.
- Software testing, deployment, and maintenance.

3. AI, Data Science & Advanced Solutions

- Artificial Intelligence and Machine Learning solutions.
- Predictive analytics, Natural Language Processing (NLP), and computer vision.
- Recommendation systems and enterprise AI platforms.
- Data science solutions for business intelligence and decision-making.

4. Training & Internship Programs

- Full Stack Web Development training.
- Artificial Intelligence, Machine Learning, and Data Science programs.
- IoT and embedded systems training.
- Industry-oriented internship programs with real-time project exposure.

1.1.2 Products

Inventeron Technologies and Business Solutions LLP develops advanced technology-driven solutions to enhance efficiency and enable digital transformation across industries such as mobility, manufacturing, healthcare, and enterprise systems. The company builds scalable and user-friendly web and enterprise applications capable of handling real-time data processing, ensuring efficient information management and improved user interaction.

It also designs and implements robust data-driven and cloud-enabled platforms that support secure data handling, analytics, and seamless integration across systems, enabling organizations to make informed decisions and improve operational efficiency.

Inventeron specializes in IoT-based solutions and embedded systems that utilize connected devices and intelligent sensors to collect, process, and analyze real-time data, reducing manual intervention and enhancing automation accuracy. In addition, the company provides EV system integration, hardware-software interfacing, and advanced engineering solutions to optimize system performance and accelerate innovation.

The organization further delivers Artificial Intelligence, Machine Learning, and Data Science solutions, including predictive analytics, computer vision, and Natural Language Processing (NLP), to support intelligent automation and business insights. It also develops efficient backend systems and data platforms for streamlined data storage, processing, and retrieval..

Moreover, Inventeron supports skill development through structured training and internship programs that emphasize practical learning and real-world project experience. Overall, its solutions and initiatives contribute to improved automation, intelligent data management, and technological advancement across modern industries.

1.2 Company Profile

Company Name: Inventeron Technologies and Business Solutions LLP

Office Address: #322, 3rd Floor, Sankalp Business Bay, Bangalore, India

Website: www.inventeron.com

Year Stand Up: 2013

Industry: Engineering Services, Software Development, Embedded Systems, Internet of Things (IoT), Electric Vehicle (EV) Technologies, Artificial Intelligence, Data Science, and Machine Learning Solution

CHAPTER 2

OBJECTIVES OF INTERNSHIP

- To develop a full-stack ride-sharing web application using the MERN stack (MongoDB, Express.js, React, Node.js).
- To design and implement a user-friendly interface for passengers and drivers with separate dashboards and role-based access.
- To build core functionalities including user authentication, ride posting, ride searching, and seat-based booking system.
- To integrate MongoDB database for storing user data, ride details, and booking information efficiently.
- To design and develop RESTful APIs for seamless communication between frontend and backend systems.
- To implement a scalable and efficient system for managing rides, bookings, and user interactions.

CHAPTER 3

LEARNING EXPERIENCE

During the development of the CO-GO Ride Sharing application, I gained valuable hands-on experience in building a complete full-stack web application using modern web technologies. I worked on both frontend and backend, which helped me understand how real-world applications are designed and developed. On the frontend, I learned how to create responsive and user-friendly interfaces using React.

I ensured smooth interaction for both passengers and drivers while performing operations such as registration, login, ride searching, and booking. On the backend, I developed practical skills in Node.js and Express.js. I handled routing, processed user requests, and built RESTful APIs to manage authentication, ride posting, and booking functionalities.

I also gained experience in integrating MongoDB as a database to store user information, ride details, and booking records efficiently. This helped me understand data management and how frontend and backend communicate using tools like Axios. Additionally, working on this project improved my problem-solving, debugging, and system design skills. I faced real-time development challenges and learned how to resolve them effectively.

Overall, this project provided me with a strong understanding of full-stack development, including frontend design, backend logic, and database integration, preparing me for real-world software development challenges.

CHAPTER 4

INTERNSHIP PROJECT OVERVIEW

4.1 Introduction to the Project Undertaken During the Internship

During my internship at Inventeron Technologies, I developed a full-stack web application titled “CO-GO Ride Sharing System.” The project is designed to connect passengers with drivers traveling on the same route and provide an efficient, cost-effective transportation solution through a carpooling platform.

In modern urban environments, transportation challenges such as high travel costs, traffic congestion, and inefficient vehicle usage are common issues. Traditional travel methods often lack coordination between passengers and available rides, leading to wasted resources and inconvenience. This project addresses these problems by providing a digital platform where users can easily share rides based on location, date, and availability.

The system provides a user-friendly web interface where users can register as either passengers or drivers, log in securely, and access role-based dashboards. Drivers can post ride details such as source, destination, travel date, price, and available seats, while passengers can search for available rides and book seats accordingly. The platform also manages booking data and ensures smooth interaction between users.

The application is developed using the **MERN stack**, where React is used for frontend development to create responsive user interfaces, and Node.js with Express.js is used for backend development to handle APIs and business logic. MongoDB is integrated as the database for efficient storage and retrieval of user data, ride details, and booking information.

Overall, this project provides a scalable and practical solution for ride-sharing and demonstrates the real-world application of full-stack web development. It enhances user convenience, promotes resource optimization, and reflects the effective integration of frontend, backend, and database technologies in solving real-world transportation problems.

4.2 System Requirements

The system enables users to access a web-based ride-sharing platform where passengers and drivers can interact efficiently. Users can register, log in, and perform role-based

operations such as posting rides, searching for available trips, and booking seats. The application ensures smooth interaction between the frontend, backend, and database components.

The system provides real-time functionalities such as ride creation, ride search, and seat booking, ensuring a seamless user experience. It also manages user authentication, ride details, and booking information effectively using MongoDB for efficient data storage and retrieval. The integration of frontend and backend through REST APIs ensures reliable and fast communication within the system.

4.2.1 Functional Requirements

- Allow users to register and log in as either a Passenger or a Driver (Host).
- Enable drivers to post ride details such as source, destination, travel date, price, and available seats.
- Allow passengers to search for available rides based on source, destination, and date.
- Provide functionality for passengers to book seats in a selected ride.
- Store user information, ride details, and booking data in MongoDB for efficient management.
- Display available rides dynamically to passengers and update seat availability after booking.
- Ensure proper validation of user inputs during signup, login, ride posting, and booking to prevent errors

4.2.2 Non-Functional Requirements

- **Usability:** The system should provide a simple, intuitive, and user-friendly interface for passengers and drivers to easily register, log in, search rides, and book seats.
- **Performance:** The system should handle ride searches, bookings, and data retrieval efficiently, ensuring quick response times and smooth user experience.
- **Reliability:** The system should handle invalid inputs and errors properly to ensure accurate and stable operation.
- **Scalability:** The system should be capable of handling an increasing number of users, rides, and bookings without performance degradation.
- **Security:** The system should ensure secure handling of user credentials and data, including safe storage of user information and protection against unauthorized access.

4.3 Software Requirements

The development of the CO-GO Ride Sharing System requires modern web development technologies for frontend design, backend processing, and database management. The frontend of the application is developed using React.js, which provides a responsive and dynamic user interface for both passengers and drivers.

The backend is built using Node.js and Express.js, which handle routing, API creation, and business logic for user authentication, ride management, and booking functionalities. MongoDB is used as the database for storing user details, ride information, and booking records efficiently.

For communication between the frontend and backend, Axios is used to send and receive HTTP requests. The application is developed using tools such as Visual Studio Code, and testing is performed using web browsers like Google Chrome

4.3.1 Node.js & Express.js

Node.js and Express.js are used as the backend framework for developing the web application. They handle server-side operations, manage routing, process user requests, and provide RESTful APIs for functionalities such as user authentication, ride posting, and booking.

4.3.2 React.js

React.js is used for developing the frontend of the application. It helps in creating dynamic, responsive, and user-friendly interfaces for passengers and drivers, enabling smooth interaction with features like signup, login, ride search, and booking.

4.3.3 MongoDB

MongoDB is used as the database for storing application data, including user details, ride information, and booking records. It enables efficient data storage, retrieval, and management, supporting scalability and flexibility.

4.3.4 Libraries Used

Various libraries are used to enhance functionality, such as:

- Axios for handling HTTP requests between frontend and backend
- Mongoose for connecting and interacting with MongoDB

- Express for backend routing and API development
- React Router for navigation between different pages
- CORS for enabling secure communication between frontend and backend

4.3.5 Development Tools

- Visual Studio Code for code development and debugging.
- Google Chrome for testing and debugging the application.
- MongoDB Compass for database visualization and management.

4.4 DESIGN

The CO-GO Ride Sharing System is designed using a modular and layered architecture to ensure scalability, efficiency, and maintainability. The system mainly consists of three major layers: frontend, backend, and database.

The frontend layer is developed using React.js, providing a dynamic, responsive, and user-friendly interface. It allows users to register, log in, search for rides, post ride details, and book seats. The interface ensures smooth interaction for both passengers and drivers through role-based dashboards.

The backend layer is implemented using Node.js and Express.js, which handle user requests, process input data, and manage application logic. It provides RESTful APIs for functionalities such as authentication, ride creation, ride searching, and booking. It also manages routing between different modules of the application.

The database layer uses MongoDB to store all application data, including user information, ride details, and booking records. This enables efficient data storage, retrieval, and management, ensuring scalability and fast performance.

The database layer uses MongoDB to store all application data, including user information, ride details, and booking records. This enables efficient data storage, retrieval, and management, ensuring scalability and fast performance.

CHAPTER 5

INTERNSHIP PROJECT IMPLEMENTATION

5.1 Introduction

This chapter explains the implementation details of the CO-GO Ride Sharing System developed during the internship. It describes how different modules of the system were designed and integrated using full-stack web development technologies. The implementation focuses on creating a scalable, efficient, and user-friendly platform for ride sharing between passengers and drivers.

The system includes key functionalities such as user authentication, ride posting, ride searching, and seat booking. Each module is developed to ensure smooth interaction and real-time data handling between users and the system.

The implementation also highlights how the frontend, backend, and database work together to deliver complete system functionality. React is used for building the user interface, Node.js and Express.js handle backend operations, and MongoDB is used for efficient data storage and management.

Overall, this chapter provides a detailed understanding of how the system components are integrated to create a reliable and functional ride-sharing application.

5.2 Implementation

The system was implemented using a full-stack approach. The frontend was developed using React.js to provide a dynamic and interactive user interface for both passengers and drivers. The backend was built using Node.js and Express.js, which handle user requests, process data, and manage application logic through RESTful APIs. MongoDB is used as the database to store user information, ride details, and booking data efficiently.

The system is divided into modules such as user authentication (signup/login), ride posting, ride searching, and booking management. Each module is designed independently and integrated to ensure smooth functioning of the application and seamless interaction between users and the system.

5.2.1 User Authentication Module

The authentication module allows users to register and log in as either a passenger or a driver. During registration, users provide details such as name, email, password, and role. The system validates the input data and stores it securely in MongoDB. During login, user credentials are verified, and based on the role, users are redirected to their respective dashboards.

5.2.2 Ride Posting Module (Driver Module)

The ride posting module enables drivers to create ride offers by entering details such as source location, destination, travel date, number of available seats, and price per seat. The entered data is validated and stored in MongoDB. These ride details are then made available for passengers to search and book.

5.2.3 Ride Search Module (Passenger Module)

The ride search module allows passengers to search for available rides based on source, destination, and date. The system retrieves matching ride data from MongoDB and displays it dynamically. This helps passengers find suitable rides quickly and efficiently.

5.2.4 Booking Module

The booking module allows passengers to select a ride and book the required number of seats. The system updates seat availability accordingly and stores booking details in the database. This ensures proper management of ride capacity and booking records.

5.2.5 Dashboard Module

The dashboard module provides an overview of system activities. Drivers can view their posted rides, and passengers can view available rides and booking details. It enhances user experience by presenting data in an organized and user-friendly manner.

5.3 Flow Chart

The flowchart shown in Figure 5.1 represents the workflow of the CO-GO Ride Sharing System. The process begins when the user opens the web application and proceeds to either sign up or log in by providing valid credentials. The system then validates the user details such as email, password, and role to ensure secure access.

Once authentication is successful, the system identifies the role of the user, which can be either a Host (Driver) or a Passenger. Based on this role, the user is redirected to the respective dashboard, enabling role-based functionality within the application.

In the Host (Driver) flow, the user accesses the host dashboard and creates a ride by entering details such as source, destination, travel date, price, and number of available seats. After submission, the ride information is stored in the MongoDB database, making it available for passengers to view and book.

In the Passenger flow, the user accesses the passenger dashboard and searches for available rides based on selected criteria such as route and date. The system retrieves matching rides from the database and displays them to the user. The passenger can then select a ride and proceed to book a seat. Once the booking is confirmed, the system updates the available seats in the database to maintain accurate ride information.

After completing either ride posting or ride booking, the system successfully processes the request and displays the outcome to the user. All relevant data, including ride details and booking information, is managed efficiently through the database. Finally, the process ends after completing the required operation.

Additionally, the system ensures data consistency and real-time updates by synchronizing user actions with the backend and database. Whenever a ride is created or booked, the corresponding information is instantly updated in MongoDB, ensuring that passengers always view accurate and up-to-date ride availability. This improves reliability and prevents issues such as overbooking or data mismatch.

Furthermore, the architecture of the system supports scalability and future enhancements. The modular separation between frontend, backend, and database allows easy integration of advanced features such as online payment systems, real-time GPS tracking, and rating or review mechanisms. This makes the system adaptable to real-world applications and capable of handling a growing number of users efficiently.

Overall, the flowchart clearly illustrates the sequence of operations, role-based decision-making, and interaction between frontend, backend, and database components in the CO-GO Ride Sharing system, demonstrating an efficient and structured workflow for ride management and booking.

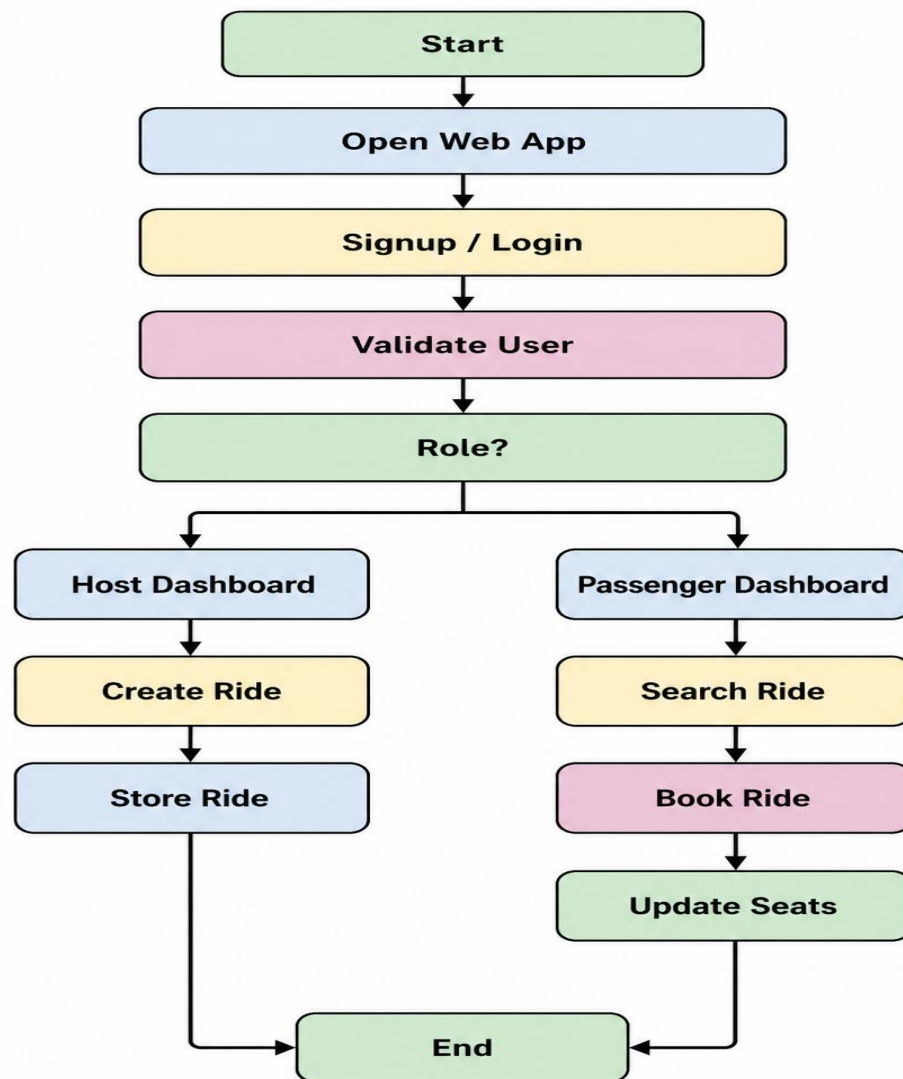


Figure 5.1: Flow Chart Diagram of the Project

5.4 Algorithm Used

The system follows the following workflow:

1. User registers or logs in to the system as a passenger or driver.
2. Input data (user details, ride details, or search parameters) is entered through the web interface.
3. The frontend sends the data to the backend using API requests (Axios).
4. The backend processes the request using Node.js and Express.js.
5. For drivers, ride details are stored in MongoDB
6. For passengers, the system retrieves matching rides from MongoDB based on search criteria.

7. For passengers, the system retrieves matching rides from MongoDB based on search criteria.
8. The system updates seat availability and stores booking details in MongoDB.
9. The updated data is displayed to the user through the frontend interface.

5.5 Code Snippets

1. Server Setup & MongoDB Connection

```
app.post("/signup", async (req, res) => {  
  const { name, email, password } = req.body;  
  const hashed = await bcrypt.hash(password, 10);  
  await User.create({ name, email, password: hashed });  
  res.json({ message: "User created" });  
});  
  
app.post("/login", async (req, res) => {  
  const user = await User.findOne({ email: req.body.email });  
  const valid = await bcrypt.compare(req.body.password, user.password);  
  
  if (!valid) return res.status(400).json({ error: "Invalid" });  
  
  const token = jwt.sign({ id: user._id }, "secret");  
  res.json({ token });  
});
```

2. Authentication (Signup + Login with JWT)

```
app.post("/create-trip", auth, async (req, res) => {  
  const trip = new Trip({ ...req.body, driver: req.user.id });  
  await trip.save();  
  res.json({ message: "Trip created" });  
});  
  
app.post("/book-trip", auth, async (req, res) => {  
  const trip = await Trip.findById(req.body.tripId);  
  trip.seats -= req.body.seats;  
  await trip.save();  
  
  await Booking.create({  
    tripId: req.body.tripId,
```

```
    userId: req.user.id
  });

  res.json({ message: "Booked" });
});
```

3. Trip & Booking APIs

```
app.post("/create-trip", auth, async (req, res) => {
  const trip = new Trip({ ...req.body, driver: req.user.id });
  await trip.save();
  res.json({ message: "Trip created" });
});
```

```
app.post("/book-trip", auth, async (req, res) => {
  const trip = await Trip.findById(req.body.tripId);
  trip.seats -= req.body.seats;
  await trip.save();
```

```
  await Booking.create({
    tripId: req.body.tripId,
    userId: req.user.id
  });
```

```
  res.json({ message: "Booked" });
});
```

4. Authentication (Signup + Login)

```
const signup = async () => {
  await API.post("/signup", user);
};
```

```
const login = async () => {
  const res = await API.post("/login", data);
  localStorage.setItem("token", res.data.token);
};
```

5. Ride Search & Booking

```
const searchRides = async () => {
```

```
const res = await API.get(`/trips?source=${source}`);  
setRides(res.data.trips);  
};  
  
const bookRide = async () => {  
  await API.post("/book-trip", {  
    tripId,  
    seats  
  });  
};
```

5.6 Results

The CO-GO Ride Sharing System was successfully implemented and tested. The application enables smooth interaction between passengers and drivers, allowing users to register, log in, post rides, search for available trips, and book seats efficiently. All major features such as user authentication, ride management, booking functionality, and data storage using MongoDB are functioning correctly.

The system ensures efficient data handling, real-time ride updates, and a user-friendly interface for both passengers and drivers. It provides a seamless ride-sharing experience by managing ride availability and booking operations effectively. Overall, the project meets its objectives and demonstrates the effective use of full-stack web development technologies, including React, Node.js, Express.js, and MongoDB, in building a real-world ride-sharing application.

Additionally, the project highlights the importance of integrating frontend, backend, and database technologies to build a complete and functional application. It demonstrates how real-time systems can be developed to solve practical problems such as ride sharing and transportation efficiency. With future enhancements such as online payment integration, GPS tracking, and user feedback systems, the application can be further improved to provide a more advanced and scalable solution.

5.6.1 Screenshots

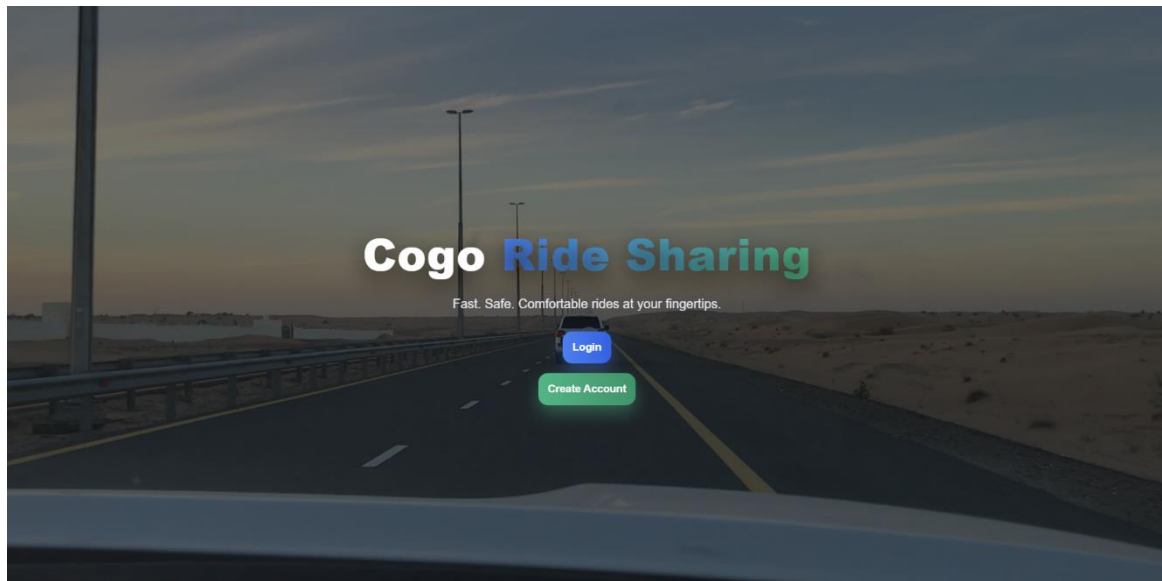


Figure 5.2: Home Page

This screen represents the landing page of Cogo Ride Sharing. It provides the initial interface for users to access the application and offers navigation options for login and new user registration. The page is designed to provide a simple and attractive entry point to the ride-sharing system.

It also enhances user experience by providing clear navigation and quick access to essential features. The intuitive design ensures that users can easily understand and begin using the application without any difficulty.

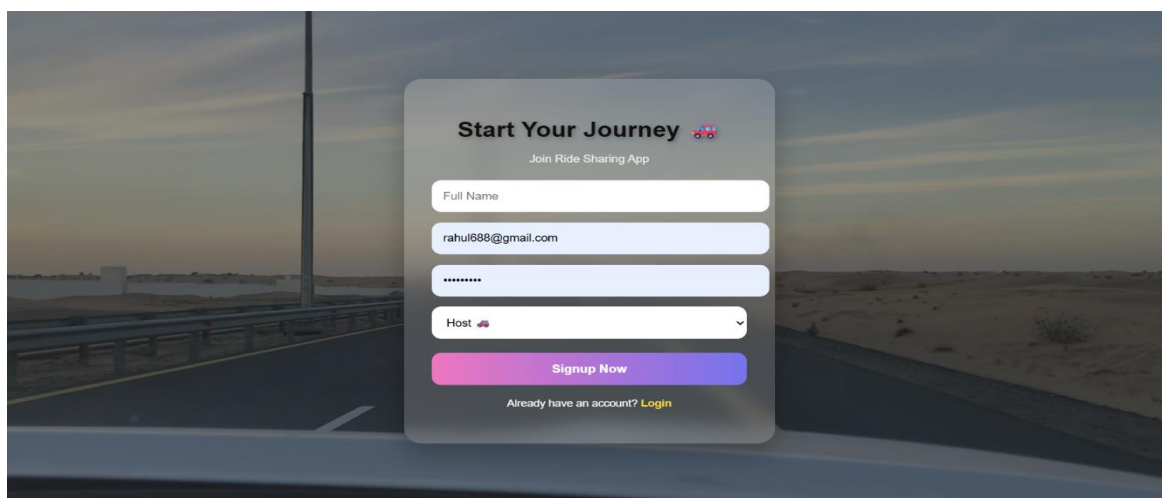


Figure 5.3: Sign Up Page

This screen shows the user registration interface of Cogo Ride Sharing. New users can enter their personal details such as name, email, password, and role to create an account. The entered information is sent to the backend and stored in the MongoDB database after successful validation.

The form ensures proper input validation to prevent errors and maintain data accuracy. It provides a simple and user-friendly interface that allows users to quickly create an account and access the application features.

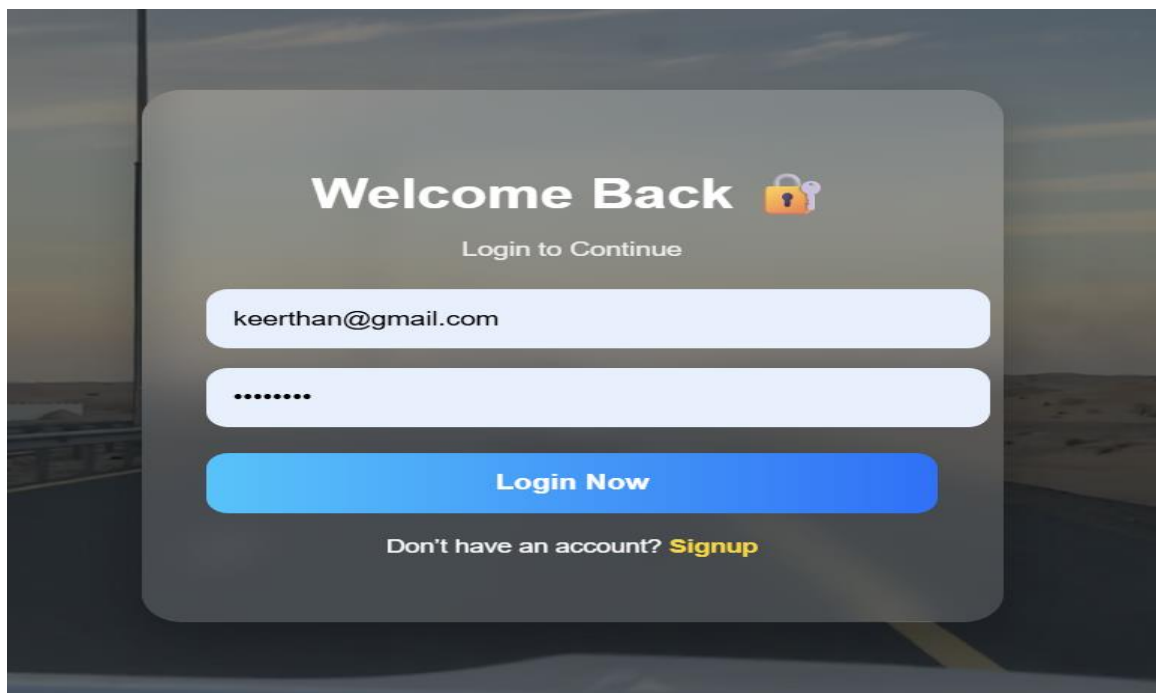
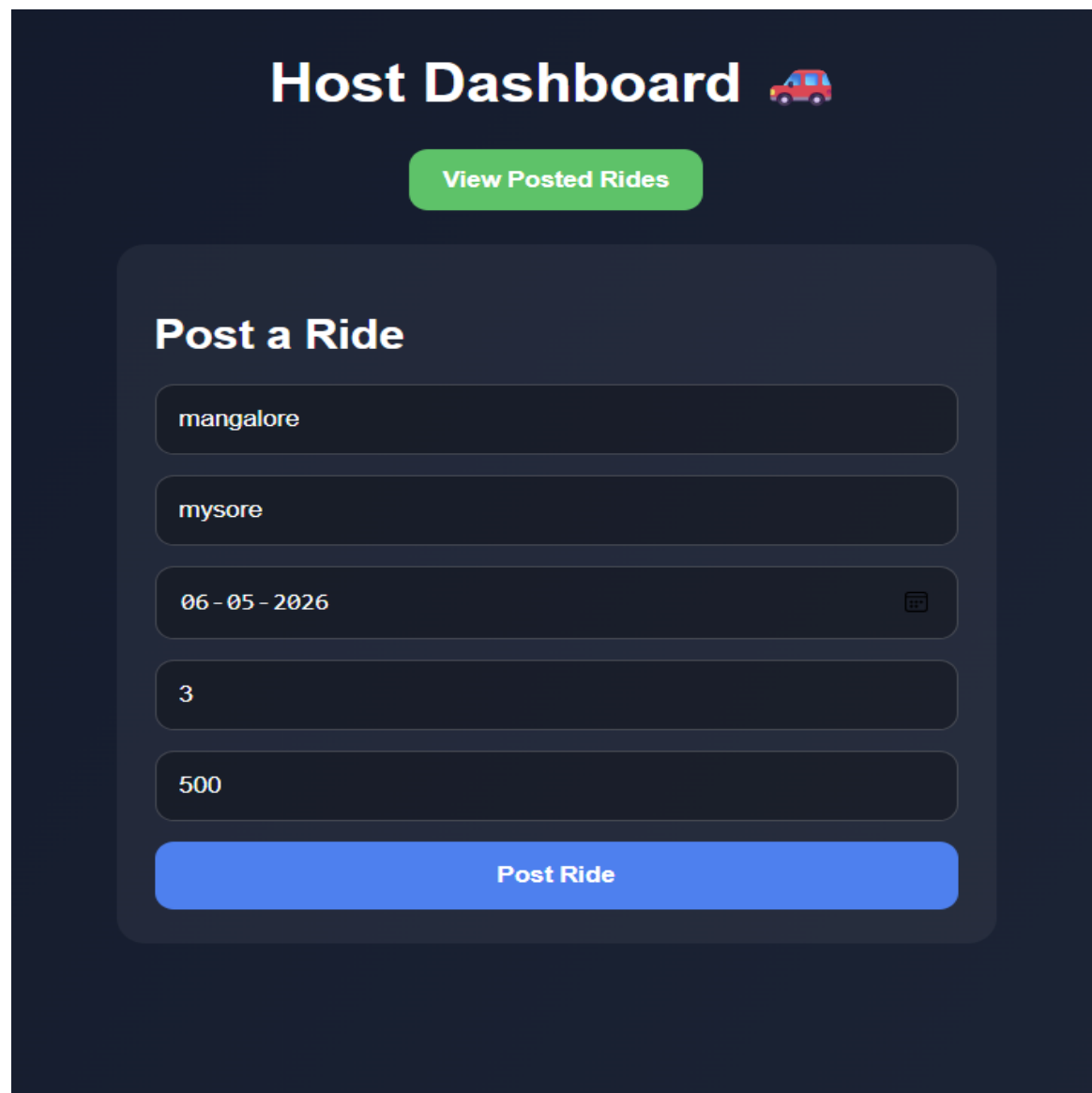


Figure 5.4: Login Page

This screen represents the login interface of Cogo Ride Sharing, where registered users enter their email and password to access the system. After successful authentication, a JSON Web Token (JWT) is generated. The user is then redirected to the appropriate dashboard based on their role.



The image shows a 'Host Dashboard' for a ride-sharing application. At the top, the title 'Host Dashboard' is displayed in white text on a dark blue background, accompanied by a small red car icon. Below the title is a green button labeled 'View Posted Rides'. The main section is titled 'Post a Ride' and contains five input fields: 'mangalore' (source), 'mysore' (destination), '06-05-2026' (date with a calendar icon), '3' (seats), and '500' (price). A large blue button labeled 'Post Ride' is positioned at the bottom of the form.

Figure 5.5: Host Dashboard

This screen represents the host dashboard of Cogo Ride Sharing, where the host can enter ride details such as source, destination, date, seats, and price. After submission, the ride is stored in the database and made available for passengers to book.

The dashboard is designed to provide an easy and efficient way for hosts to manage their rides. It ensures smooth data entry and allows hosts to quickly create and publish trips, improving the overall usability of the system.

Passenger Dashboard

My Bookings 

Search Rides

mangalore

mysore

06 - 05 - 2026 

Search Rides

Available Rides

 mangalore → mysore

 Wed May 06 2026

 Seats: 3

 ₹500

Book Now

Figure 5.6: Passenger Dashboard

This screen represents the passenger dashboard of Cogo Ride Sharing. Passengers can search for available rides by entering the source, destination, and date of travel. The system retrieves matching trips from the MongoDB database and displays them so that passengers can proceed with booking.

The dashboard provides a user-friendly interface that allows passengers to easily browse and select suitable rides. It ensures a smooth booking experience by presenting clear trip details and enabling quick decision-making for travel.

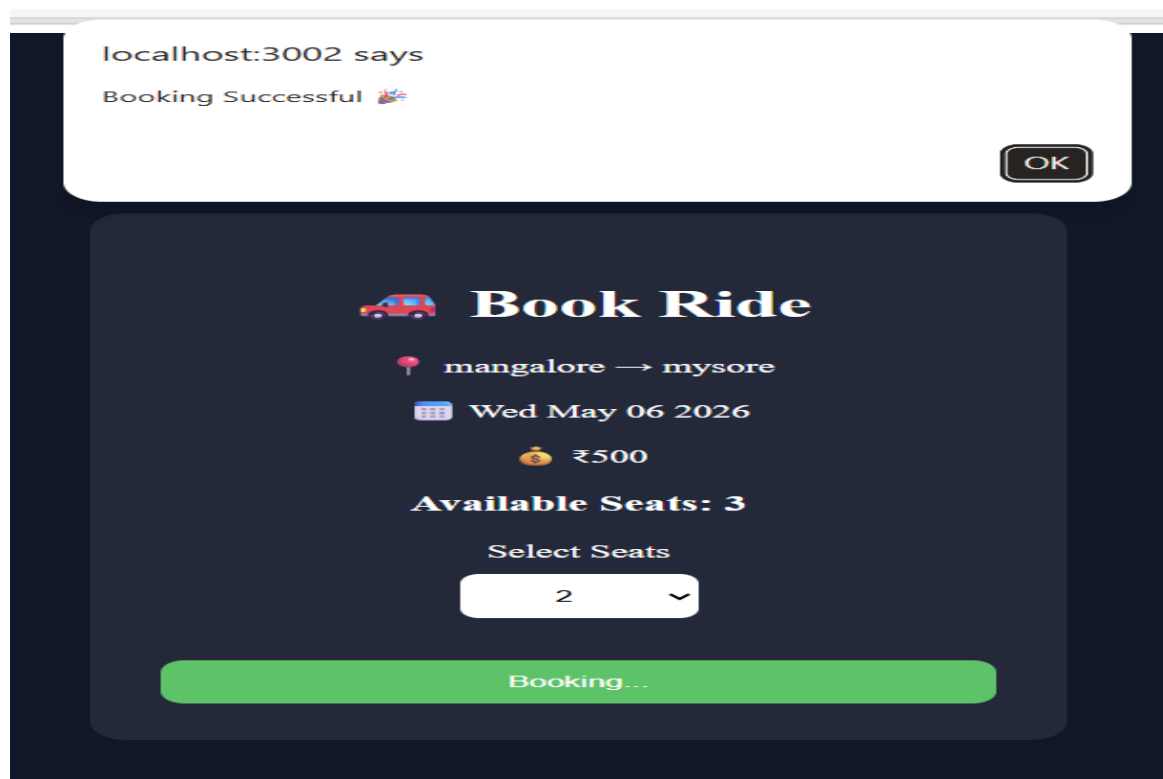


Figure 5.7: Ride Booking

This screen shows the booking confirmation page of Cogo Ride Sharing. It displays the selected trip details along with the number of seats booked by the passenger. After successful booking, the system confirms that the ride information has been stored in the MongoDB database.

CHAPTER 6

LEARNING OUTCOME

The internship and development of the **CO-GO Ride Sharing** application provided valuable learning outcomes in both technical and practical aspects of full-stack web development. One of the major learnings was understanding how to build and integrate a complete web application using the MERN stack, where React is used for the frontend, Node.js and Express.js are used for the backend, and MongoDB is used for database management. I gained hands-on experience in designing responsive and user-friendly interfaces for passengers and drivers, along with implementing role-based dashboards and navigation

I also developed knowledge in backend development, API creation, and database integration. Multiple core features such as user authentication, ride posting, ride searching, and booking were implemented and connected with the frontend using Axios. Additionally, I gained experience in managing data efficiently using MongoDB, including storing and retrieving user details, ride information, and booking records.

Another important learning outcome was understanding how frontend, backend, and database components work together in a real-world application. The project also improved my problem-solving and debugging skills while handling issues related to routing, API integration, and data flow between modules. Overall, this internship helped me bridge the gap between theoretical knowledge and practical implementation, preparing me for real-world challenges in web development, database management, and application integration

CHAPTER 7

ASSESSMENT & CONCLUSION

7.1 Assessment of Internship

The internship at Inventeron Technologies was highly valuable and provided practical exposure to real-world full-stack web development. It offered an opportunity to work on a complete project, starting from frontend design to backend development and database integration. The guidance and support received during the internship helped in understanding industry practices, coding standards, and project development methodologies.

Working on the CO-GO Ride Sharing System improved my technical skills in React-based frontend development, Node.js and Express.js backend development, and MongoDB database integration. It also enhanced my ability to design RESTful APIs, manage user authentication, and implement real-time features such as ride posting, searching, and booking. Additionally, the internship helped improve my problem-solving, debugging, time management, and adaptability skills, which are essential in a professional environment.

7.2 Conclusion

The CO-GO Ride Sharing System successfully achieved its objective of developing an efficient and user-friendly web application for ride sharing. The system provides key features such as user authentication, ride posting, ride searching, and booking functionality, along with efficient data storage using MongoDB.

The use of modern technologies such as React, Node.js, Express.js, and MongoDB ensured that the system is scalable, efficient, and capable of handling real-world scenarios. The project demonstrates the effective integration of frontend, backend, and database components to build a complete full-stack application.

In conclusion, this internship and project provided valuable hands-on experience in web development and system integration. With further enhancements such as online payment systems, real-time tracking, and advanced features, the application can be expanded into a more powerful and user-centric ride-sharing platform.

REFERENCES

- [1] Tilkov, S., & Vinoski, S. (2010). Node.js: Using JavaScript to Build High-Performance Network Programs. *IEEE Internet Computing*, 14(6), 80–83.
- [2] Banks, A., & Porcello, E. (2017). *Learning React: Functional Web Development with React and Redux*. O'Reilly Media.
- [3] Richardson, L., & Ruby, S. (2007). *RESTful Web Services*. O'Reilly Media.
- [4] MongoDB Inc. (2024). *MongoDB Database Documentation*. Retrieved from <https://www.mongodb.com/docs/>
- [5] Express.js. (2024). *Fast, Unopinionated, Minimalist Web Framework for Node.js*. Retrieved from <https://expressjs.com>